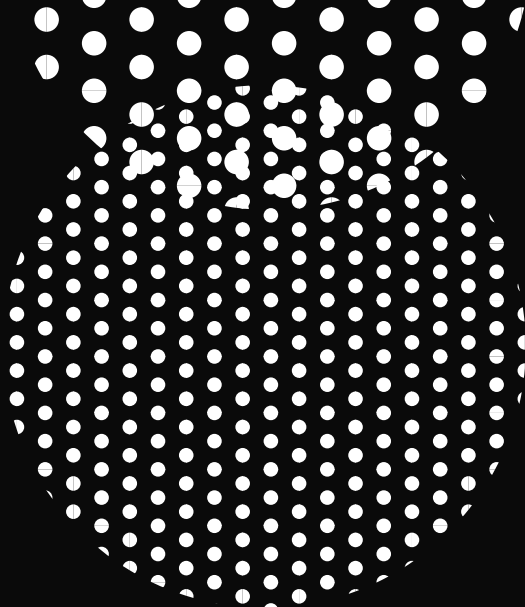
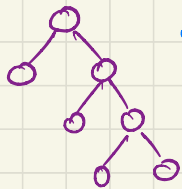




ADABOOST

In RF, each time you make a tree, you make a full-sized tree



some trees might be bigger than others but there's no predetermined max depth

- ↳ each tree has an equal vote on the final classification
- ↳ each decision tree is made independently of others

In a forest of trees made with AdaBoost, (trees are usually just a node and two leaves)



weak learners/stump



- ↳ some stumps get more say in the final classification
- ↳ order is important: errors that the n th stump makes affects the $(n+1)$ th stump
- ↳ stump: not great at making accurate classifications: can only use one variable

Scenario: Predict if patient has heart disease based on chest pain, blocked artery status, and weight

Steps:

- ↳ give each sample a weight that indicates how important it is to be correctly classified
- note: at start, all samples get the same weight $1 / \# \text{ samples}$

- ↳ make 1st stump by selecting the parameter that does best

★ Find variable that best classifies heart disease using data, calculating the gini weight and picking variable with least gini weight for the first stump

★ Determine how much say the stump has in the final

classification: Total Error (TE) is sum of the weights associated with the incorrectly classified samples

$$\text{amount of ray} = \frac{1}{2} \log \left(\frac{1-TE}{TE} \right)$$

when a stump does a good job and TE is small, amount of ray is a big +ve number

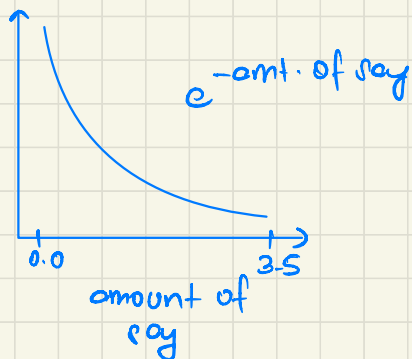
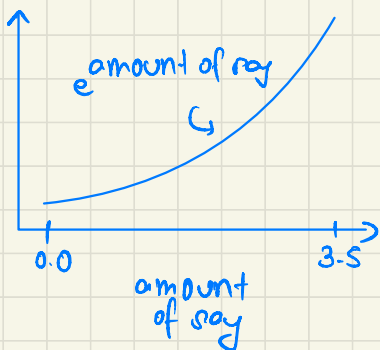


★ How to modify the weights so that the next stump will take the errors that the current stump made into account?

- ↑ the sample weight of the incorrectly classified sample
- ↓ the sample weights of the others

$$\text{net sample wt.} = \text{sample wt.} \times e^{\text{amount of ray}}$$

$$\text{net} = \text{sample wt.} \times e^{-\text{amt. of ray}}$$



★ Normalize by dividing each sample wt. by the total sample wt.

Advantages:

- High degree of interpretability
- High accuracy + easy to train

Disadvantages:

- sensitive to noisy data and outliers
- computationally expensive if dataset is big
- requires careful parameter tuning to reach optimal performance